

AI ASSISTED CODING

SAI SREEYESH

2303A510H8

BATCH – 03

07 – 04 – 2026

ASSIGNMENT – 20.2

Lab 20 : Security Testing : Identifying Vulnerabilities in AI – Generated Code.

Task – 01 : Password Security Enhancement.

Prompt : Generate a Python user registration and login script that stores usernames and passwords.

Code & Output :



The screenshot shows a code editor interface with a menu bar (File, Edit, View, Insert, Running, Tools, Help) and a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. The code is titled 'TASK 01' and is written in Python. It uses the 'bcrypt' library for password hashing. The code defines a 'register' function that takes 'username' and 'password' as arguments, hashes the password, and stores it in a dictionary. It also defines a 'login' function that checks the username and password against the stored hashes. The code is executed, and the output is shown at the bottom: 'Login Successful' and 'Invalid Credentials'.

```
import bcrypt

users = {}

def register(username, password):
    hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
    users[username] = hashed

def login(username, password):
    if username in users and bcrypt.checkpw(password.encode(), users[username]):
        return "Login Successful"
    return "Invalid Credentials"

register("admin", "password123")
print(login("admin", "password123"))
print(login("admin", "wrong"))
```

Output:

```
... Login Successful
Invalid Credentials
```

Explanation :

Plain text passwords are unsafe because they can be easily exposed in data breaches. Hashing converts passwords into irreversible values, improving security. bcrypt adds salting, preventing attacks like rainbow tables. The updated script securely verifies login without storing real passwords.

Task – 02 : Secure API Key Management.

Prompt : Generate a Python script that fetches data from an API using an API key.

Code :



```
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all
TASK 02
[5] ✓ Os
import os
import requests

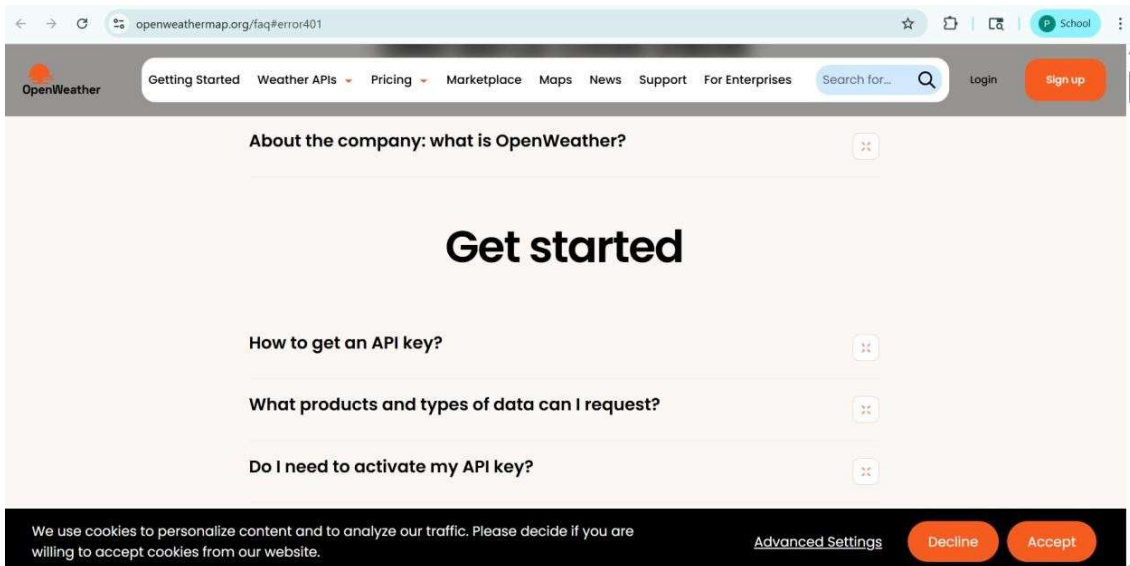
api_key = os.getenv("API_KEY")

url = f"https://api.openweathermap.org/data/2.5/weather?q=London&appid={api_key}"
response = requests.get(url)

print(response.json())

*** {'cod': 401, 'message': 'Invalid API key. Please see https://openweathermap.org/faq#error401 for more info.'}
```

Output :



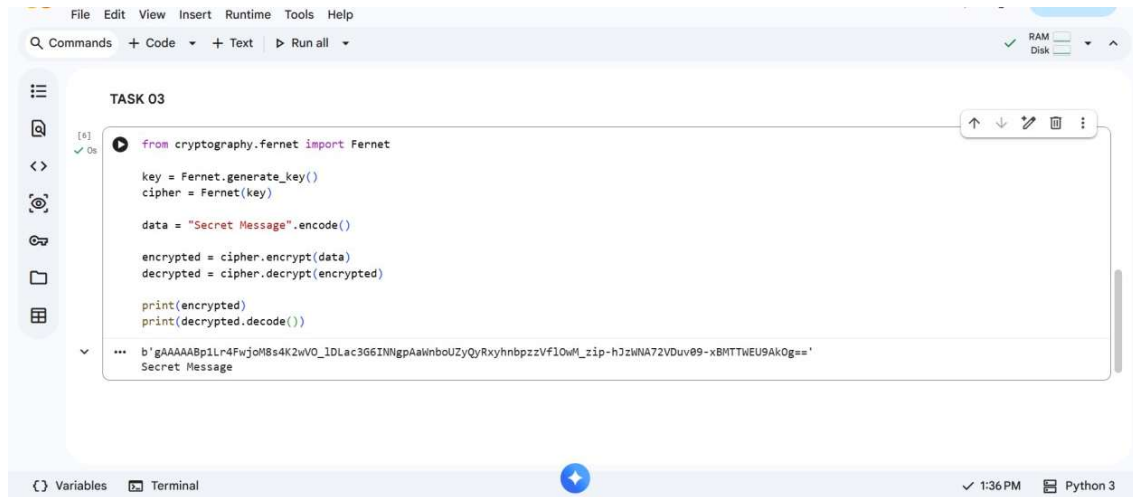
Explanation :

Hardcoding API keys exposes them in source code and version control. Environment variables securely store sensitive data outside the code. This prevents unauthorized access and improves deployment security. The script still works normally while keeping credentials hidden.

Task – 03 : Weak Encryption Detection.

Prompt : Generate a Python encryption-decryption program using a simple method.

Code & Output :

The screenshot shows a code editor window titled 'TASK 03'. The code is in Python and uses the 'Fernet' module from 'cryptography'. It generates a key, creates a cipher object, encodes the string 'Secret Message', encrypts it, and then decrypts it back to the original string. The output is displayed in a terminal-like window at the bottom, showing a long base64-encoded string followed by the decoded 'Secret Message'.

```
from cryptography.fernet import Fernet

key = Fernet.generate_key()
cipher = Fernet(key)

data = "Secret Message".encode()

encrypted = cipher.encrypt(data)
decrypted = cipher.decrypt(encrypted)

print(encrypted)
print(decrypted.decode())
```

b'gAAAAABp1Lr4Fwj0M8s4K2wV0_IDLac3G61NNgpAaWnboUzYqRxyhnbppzVf10wM_zip-h3zWNA72VDuv09-xBMTTWEU9AkOg=='
Secret Message

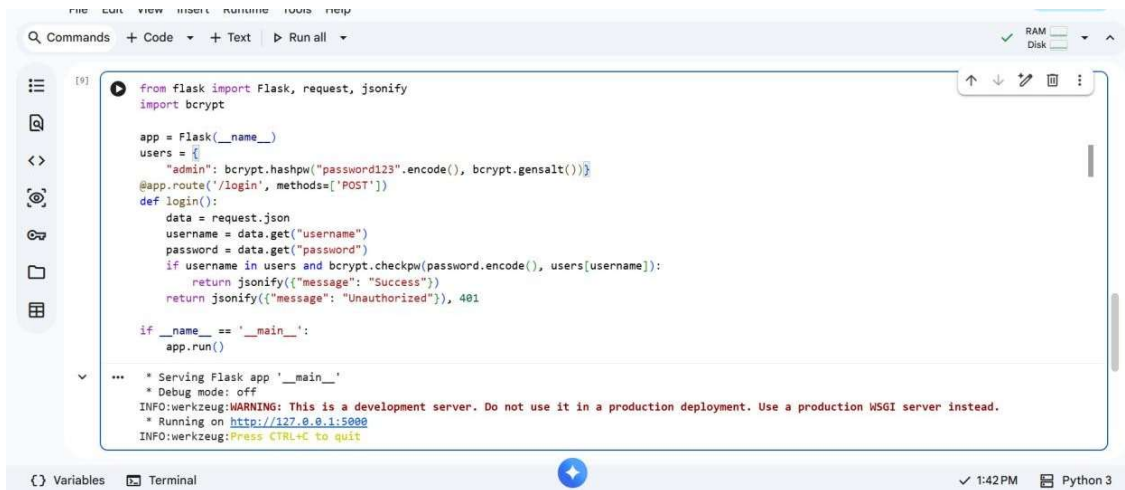
Explanation :

Basic encoding like base64 is reversible and not secure encryption. Fernet provides strong symmetric encryption using secure algorithms. Data cannot be decrypted without the correct key. This ensures confidentiality and protects sensitive information.

Task – 04 : Real Time Application : Security Review of AI – Generated Web Service.

Prompt : Generate a simple Flask API for user login.

Code & Output :



```
[9] from flask import Flask, request, jsonify
import bcrypt

app = Flask(__name__)
users = {
    "admin": bcrypt.hashpw("password123".encode(), bcrypt.gensalt())
}
@app.route('/login', methods=['POST'])
def login():
    data = request.json
    username = data.get("username")
    password = data.get("password")
    if username in users and bcrypt.checkpw(password.encode(), users[username]):
        return jsonify({"message": "Success"})
    return jsonify({"message": "Unauthorized"}), 401

if __name__ == '__main__':
    app.run()

... * Serving Flask app '__main__'
* Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
```

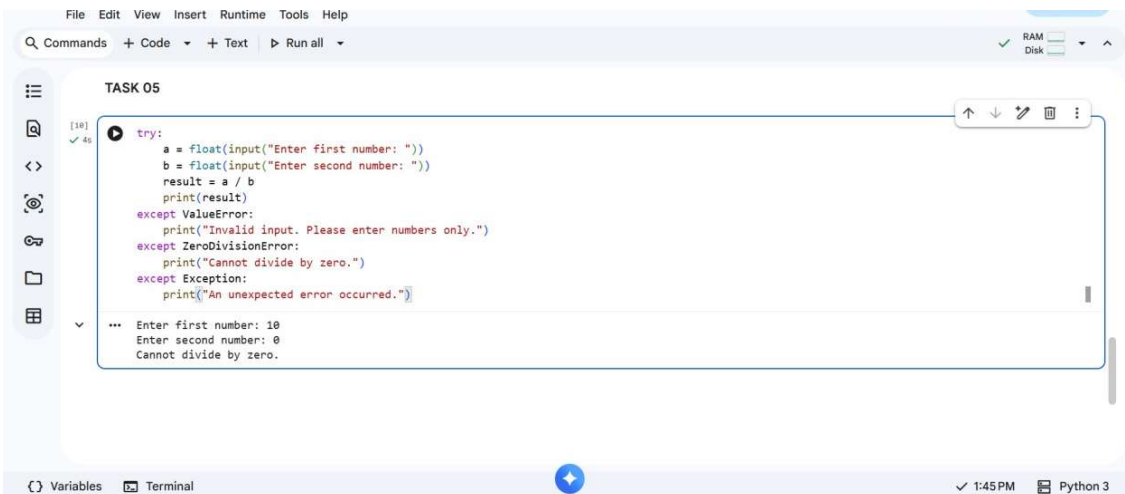
Explanation :

Common vulnerabilities include plain-text passwords, no validation, and insecure responses. bcrypt ensures passwords are securely stored and verified. Input is handled safely using JSON parsing. The API now reduces authentication risks and improves overall security.

Task – 05 : Exception Handling Improvement.

Prompt : Generate a Python program that takes two numbers and performs division.

Code & Output :



```
TASK 05

[10] try:
✓ 4s a = float(input("Enter first number: "))
      b = float(input("Enter second number: "))
      result = a / b
      print(result)
except ValueError:
    print("Invalid input. Please enter numbers only.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
except Exception:
    print("An unexpected error occurred.")

... Enter first number: 10
Enter second number: 0
Cannot divide by zero.
```

Explanation :

Programs without exception handling may crash during runtime errors. Invalid inputs and division by zero are common issues. try-except blocks

ensure smooth execution and prevent crashes. Users receive clear error messages instead of program failure.